Lab 3

1. Create a class called Person that represents a person. A person should include four pieces of information as instance variables.
    - Name of type String
    - Salary of type double
    - Is working of type Boolean
    - Monthly expenses of type double
    - Default salary of type double with constant value (1000)

    Your class should have two constructors that initialize the four instance variables.
    - First one takes one string argument. Set the person's name with this name and set the non-constant fields with their default values (monthly expenses and salary with 0 and is working with false).
    - Second one takes the four arguments and sets the person's non constant fields correspondingly.

    Provide a set and a get method for each instance variable. In addition, provide a method named net income that returns a double. The net income is the difference between monthly expenses and salary.

    Your code must follow the following restrictions.

    - If the person isn't working, the salary should be set to 0, otherwise it should be positive (if this is not the case set the violating variable to valid state ex: if you set is working to true and value of salary isn't positive set it to default salary inside the is working setter).

    Note: Please take into your consideration the above restrictions in any place in your code that modifies any instance variable.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed
Eng. Ahmed Ashraf
Eng. Miar Mamdouh
Eng. AbdElaziz Mohamed
Eng. Mazen Sallam
Eng. Shams Zayan
Eng. Abdelrahman Wael
Eng. Mohamed Zaytoon
Eng. Muhannad Bashar

2. Develop a banking system simulation by creating a class called BankAccount. This class should represent a bank account with attributes for the account number, the account holder's name, and the current balance.

The class should provide methods to allow users to deposit money into the account, withdraw money from the account, and check the current balance.

The design should ensure that the account maintains proper state and handles basic banking operations.

Example Scenario:

- A user creates a new bank account with an account number, their name, and an initial balance of 0.
- The user deposits money into their account.
- The user attempts to withdraw money, ensuring they cannot withdraw more than their current balance.
- The user checks their account balance and retrieves account details.

  Example Input:

  i. Create a new bank account:
  1. Account Number: "123456"
  2. Account Holder Name: "John Doe"
  3. Initial Balance: 0.0

  ii. Perform operations:
  1. Deposit 1000.0 EGP.
  2. Withdraw 500.0 EGP.
  3. Check Balance.
  4. Withdraw 600.0 EGP (should fail due to insufficient funds).
  5. Check Balance again.

Example Output:

1. Account Creation:
   - Account Number: 123456
   - Account Holder Name: John Doe
   - Initial Balance: 0.0 EGP
2. After Deposit:
   - Deposit Amount: 1000.0 EGP
   - New Balance: 1000.0 EGP
3. After Withdrawal:
   - Withdrawal Amount: 500.0 EGP
   - New Balance: 500.0 EGP
4. Checking Balance:
   - Current Balance: 500.0 EGP
5. Attempt to Withdraw More Than Balance:
   - Withdrawal Attempt: 600.0 EGP
   - Output: "Insufficient funds. Withdrawal failed."
6. Final Balance Check:
   - Current Balance: 500.0 EGP

Additional Considerations:

- The class should handle edge cases such as depositing negative amounts or withdrawing amounts greater than the balance, providing appropriate messages for each case.
- Make sure to adhere to OOP principles like encapsulation, ensuring that attributes are private and accessible only through methods.
- The program should be structured in a way that encourages clean and maintainable code, allowing for easy expansion or modifications in the future.

Required:

1. You are required to answer all the above questions and deliver them in your lab time.
2. A discussion will be made with you at your lab next week on what you delivered.
3. The deadline for the delivery is your lab time.

Policies:

1. You should work individually.
2. Cheating will be severely penalized, so delivering nothing is so much better than cheating.
3. No late submission is allowed.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed
Eng. Ahmed Ashraf
Eng. Miar Mamdouh
Eng. AbdElaziz Mohamed
Eng. Mazen Sallam
Eng. Shams Zayan
Eng. Abdelrahman Wael
Eng. Mohamed Zaytoon
Eng. Muhannad Bashar